

Clench

A headband that lets a person who cannot move or speak talk, message their family and control their room, using only jaw clenches and blinks. An AI fills in the full sentence, and the system learns the person over time.

Event	Team	Date	Status
ShellHacks 2026, FIU	4 people (owners not yet assigned)	September 25, 2026	Draft for team review

How to read this doc. Part 1 says what we are building and why each choice is the right one. Every decision has a green **Why** box, and most say what we turned down. Part 2 (after the page break) is the technical plan: parts of the system, how they talk, what to build first, and where to learn the Muse. Part 3 scopes a stretch feature: using the computer through the same headband.

Part 1: Product

- 1. Summary
- 2. The problem
- 3. Who it is for
- 4. Goals and non-goals
- 5. What using it feels like
- 6. Key decisions and why
- 7. Features and priorities
- 8. The starting menu
- 9. Learning the patient
- 10. Reading body state
- 11. Safety, privacy and honesty
- 12. How we measure success
- 13. The demo
- 14. Risks and backups

- 15. Open questions

Part 2: Architecture

- A1. The system at a glance
- A2. Stack choices and why
- A3. The parts in detail (incl. pointing modes)
- A4. How the parts talk
- A5. Signal processing
- A6. Speed budget
- A7. Data we store
- A8. Code layout
- A9. Build plan: before and after the Muse
- A10. Testing
- A11. Where to learn

Part 3: Stretch feature

- B. Computer use with the Muse

1. Summary

Some people think clearly but cannot move or speak. This happens with late-stage ALS, some strokes, and high spinal cord injuries. Most of these people can still clench their jaw and blink on purpose. The Muse 2 headband picks up both of these as large, clear signals.

Clench turns those two small movements into a way to talk. A screen shows a few big options. A highlight marks one of them. If the person can turn their head a little, a normal webcam lets them move the highlight by facing an option. If not, the highlight moves from option to option by itself. Either way, when it is on the one they want, they clench. Each choice opens a smaller set of choices, so you move

down a path: *I need* → *Pain* → *Back* → *A lot*. After two or three picks, an AI writes the full sentence ("My back hurts a lot. Can you help me turn over?"), you clench to confirm, and the laptop says it out loud in English or Spanish. It can also send it as a text or place a call.

Over time, Clench learns the person. It learns how their clench looks, what they say most, and when. The things they are most likely to want move to the front, so common messages take one or two clenches instead of five.

2. The problem

- **People lose their voice but not their mind.** Speech often fails as ALS gets worse. The person still has things to say, and the people around them have to guess.
- **Today's tools are costly or tiring.** Eye-gaze systems work well but often cost thousands of dollars and need insurance approval. Picture boards need a helper to hold and point. Single-switch systems exist but are slow and do not learn.
- **Most tools do not speak the family's language.** In Miami-Dade most people speak Spanish at home. A tool that only speaks English fails the family it is meant to help.
- **Tools do not get smarter.** A person says "call Maria" five times a day and still has to dig for it every time.

3. Who it is for

Primary user: the patient

Don Luis, 72, Hialeah. ALS, now unable to speak or move his arms. Can still blink and clench his jaw. Some neck movement, which may go away. Speaks Spanish first. Lives with his daughter Maria.

What he needs: say what he needs fast, reach Maria when she is in another room, keep his own voice and way of speaking.

Secondary user: the caregiver

Maria, 44. Works from home and cares for her father. Busy, often in the next room. Not technical.

What she needs: know when he needs her, understand him without guessing, set the system up in minutes, trust it not to send things he did not mean.

What we assume about the patient: can clench the jaw and blink on purpose, can see a screen, and understands language. We do **not** assume head or hand control.

4. Goals and non-goals

Goals

1. A person with no hand or voice control can say a common need in under 10 seconds.
2. They can reach a family member by text or call without help.
3. The system works in English and Spanish from day one.
4. It gets faster the more it is used, and we can show that with a number.

5. It never speaks or sends anything the person did not confirm.

Non-goals (things we will not do, on purpose)

- **No mind reading.** We do not decode thoughts. We use muscle signals (clench, blink) that the headband catches well.
- **No "think left to move left."** The Muse has no sensors over the parts of the brain that control movement, so this cannot work.
- **No HoloLens.** It costs a day of setup, may not fit with the headband, and a screen does the job better.
- **No medical claims.** This is a prototype for communication, not a medical device. We do not diagnose anything.

5. What using it feels like

1. **Put it on.** Maria slips the headband on her father. It connects to the laptop.
2. **One minute of setup.** The screen asks him to rest for 20 seconds, then clench five times, then double blink three times. Clench now knows what his signals look like today.
3. **The board.** Six big tiles appear. The first tile is the AI's best guess for right now (at noon it might say "I'm hungry"). The others are *I need*, *People*, *How I feel*, *Say something* and *Room*.
4. **Picking.** Don Luis can still turn his head a little, so the webcam is on. He turns slightly toward *People* and it lights up. He clenches. Now he sees Maria, Carlos and the nurse, with the most likely person first. (If the webcam is off or loses his face, the highlight instead moves across the tiles by itself, about one per second, and he clenches when it reaches the one he wants.)
5. **Going back.** If he picks the wrong thing, he double blinks to go back one step.
6. **The sentence.** He picks *Maria* → *Text her*. The AI shows three short messages in his usual words, like "Mija, estoy bien, llámame a las seis." He clenches on the right one. A final screen asks "Send this?" He clenches again.
7. **It happens.** The laptop reads it aloud and texts Maria's phone.
8. **Emergency.** If he holds a clench for about a second and a half, a help alert starts counting down from five. If he did not mean it, a double blink cancels it. If not cancelled, Maria's phone gets a call.
9. **Rest.** If his eyes stay closed or he sends no input for a while, the highlight stops moving. A clench wakes it up.

6. Key decisions and why

This is the heart of the doc. Each decision says what we chose, why it is the best choice, and what we turned down.

D1 Use a menu that opens into smaller menus, not a code of gestures

The screen shows options. Picking one shows the next set of options down that path. We do not use rules like "blink twice for water, clench three times for pain."

Why: A menu is something you can see, so there is nothing to memorize. A code of gestures has to be learned, is easy to forget when tired or scared, and runs out fast: there are only so many blink and clench patterns before they blur together. A menu can hold hundreds of phrases with the same two gestures. It also lets the AI change what is shown without the person relearning anything.

Turned down: gesture codes (hard to learn, few commands, many mistakes), a full keyboard only (too slow for daily needs).

D2 Three ways to point, one setting to switch; the clench always picks

How the highlight moves is a setting called **Pointing mode**. Picking is always a clench, whatever the mode.

Mode	How it works	Best for
Scan (no webcam)	The highlight moves by itself, one tile at a time. Clench when it is on what you want.	Anyone, including people with no head movement at all. Also the safe fallback.
Webcam	A normal laptop webcam watches the face. Turning the head slightly moves the highlight to that tile, like pointing with your nose. Clench to pick.	People with some neck control. Much faster than scanning, and easy for judges to see.
Head tilt (no webcam)	The motion sensor inside the headband reads head tilt and moves the highlight. Clench to pick.	Some neck control but no camera wanted, for example privacy or a dark room.

There is also **Auto**: use the webcam when it can see a face, and drop to Scan by itself if the face is lost for about 3 seconds. Auto is the default.

Why: People are different, and ALS changes over time. Someone may point with their head this month and lose neck control next month. With a setting, they switch modes instead of switching products. Scan works for everyone because it needs only the clench. Webcam is the fastest when the person can use it, and it needs nothing but the laptop's own camera. Head tilt gives a camera-free option for people who do not want to be filmed. Because the clench is always the "pick" gesture, the person never relearns anything when the mode changes. Auto means a bad angle or poor lighting never leaves the person stuck.

Turned down: webcam only (fails when neck control goes or the lighting is bad), scan only (slow for people who can move their head), eye tracking (needs special hardware), dwell to click, meaning "look at a tile long enough to pick it" (picks things by accident while the person is just looking).

D3 Only three gestures: clench, double blink, long clench

Gesture	Meaning
Clench (short)	Pick this
Double blink	Go back / No / Cancel
Long clench (hold about 1.5 s)	Call for help (with a 5 second cancel window)

Why: Every extra gesture adds mistakes and adds something to remember. Three is the smallest set that covers moving forward, moving back and getting help. Each one is easy to tell apart from the others in the signal, so the system confuses them less.

D4 Double blink, not single blink

Why: People blink on their own about 15 to 20 times a minute. If one blink meant "go back," the system would keep jumping back by accident. Two blinks close together almost never happen by accident, so it is safe to treat them as a command.

D5 The AI suggests; the person decides

The AI never speaks or sends anything by itself. Anything that leaves the laptop (spoken aloud, text, call) needs a final clench on a "Send this?" screen.

Why: It is the person's voice. If the AI put words in their mouth even once, the family would stop trusting it. The extra clench costs about a second. It also answers the first question judges will ask: "What if the AI says the wrong thing?"

D6 The AI sorts the menu; it does not replace it

The menu is a fixed tree we write by hand. The AI decides the *order* of options and writes the final sentence. If the AI is slow or offline, the menu still works with plain phrases.

Why: A person who cannot speak cannot wait for wifi. A fixed tree means the basics always work. The AI makes it faster and more personal on top. This also makes the demo safe: if the network drops on stage, the core still runs.

Turned down: a pure chat interface where the AI makes up every option (breaks without internet, and different every time, which is confusing).

D7 The most likely choice goes first

Why: With scanning, the time to pick something depends on where it sits in the line. If the right option is first, one clench picks it with no waiting. This is where learning turns into speed: better guesses mean fewer steps. It is also how we get a clear number for the demo ("5 steps became 1").

D8 No more than six options on a screen

Why: With one second per option, six options take six seconds to pass through once. More than that and waiting gets painful. Six also keeps each tile big enough to read from a bed. "Back" is a gesture (double blink), not a tile, so it does not use up a slot or add scan time.

D9 It learns the person in three ways

(1) How their clench and blink look. (2) What they say, to whom and when. (3) Their current body state. Details in section 9.

Why: Other tools make the person adapt to the machine. This one adapts to the person. The first kind of learning fixes the biggest real problem, signals changing as the person gets tired. The second makes it fast. The third makes it feel aware. Each can be built and shown in a weekend.

D10 Body state only changes the order, never takes action

If heart rate rises and the person is restless, urgent options (pain, help, bathroom) move up. That is all it does.

Why: The headband's reading of "tense or calm" is rough and varies a lot between people. Using it to reorder is helpful when right and harmless when wrong. Using it to call for help on its own would cause false alarms, and people would learn to ignore it.

D11 Spelling is always there as a way out

A "Spell it" option lets the person build any word, letter group by letter group, with word guessing.

Why: No menu covers everything. A name, a new idea, a joke. Without a way out, the person can only say what we thought of. Spelling is slow, but it means nothing is off limits.

D12 English and Spanish from day one

Why: Most people in Miami-Dade speak Spanish at home, and older patients often prefer it. Adding a second language later means rewriting every phrase and voice. Building both in now is cheap: the AI writes in either language and the voice service has natural voices for both.

D13 Pause when the person rests

If the eyes stay closed for a while, or nothing is picked for a set time, the highlight stops. A clench starts it again.

Why: A highlight that never stops is tiring to watch. A pause also cuts accidental picks while the person rests, eats or talks with visitors. Eyes-closed shows up clearly in the brain signal, so this is one brain reading we can trust.

D14 Personal data stays on the laptop

The history of what the person said is stored in a local file. Only the current request goes to the AI, not the full history.

Why: What a patient says is private health information. Keeping it local is simpler and safer, and it is a good answer when judges ask about privacy.

D15 Build everything before the headband arrives

A stand-in lets the keyboard act as the headband: the space bar is a clench, the B key is a blink. A second stand-in plays fake or recorded brain signals. When the Muse arrives, we switch one setting.

Why: We do not have the Muse yet. If the whole app waits for it, we lose the most valuable hours. With a stand-in that sends the exact same events as the real headband, the UI, AI, learning and actions can all be built and tested now. The Muse work then shrinks to one job: turning real signals into those same events.

7. Features and priorities

P0 must work for the demo. **P1** should have. **P2** if time allows.

Priority	Feature	Done when
P0	Clench and double-blink detection	In a seated test, 9 of 10 clenches are caught and there is at most 1 false pick per 5 minutes of sitting still.

P0	One-minute calibration	A guided screen sets personal thresholds in 60 seconds or less.
P0	Scanning board with the menu tree	Up to 6 big tiles, a moving highlight, clench to pick, double blink to go back.
P0	AI sentence suggestions	After 2 to 3 picks, shows up to 3 full sentences in the chosen language within 1.5 s (or instantly from cache).
P0	Confirm then speak	Nothing is spoken or sent without a final clench. Speech plays in English or Spanish.
P0	Text a family member	A real SMS arrives on a real phone during the demo.
P0	Keyboard stand-in	The whole app runs with no headband, using keys.
P1	Webcam pointing mode	Small head turns move the highlight to the tile the person faces. Works in normal room light. Falls back to Scan if the face is lost for about 3 s.
P1	Pointing mode setting	Caregiver can switch Auto / Scan / Webcam / Head tilt at any time without restarting. The choice is saved to the profile.
P1	Learning: ranking by use and time of day	With a week of history loaded, common messages move to slot 1. On-screen counter shows steps saved.
P1	Learning: signal thresholds that keep adjusting	Thresholds update from recent signals during use without a new calibration.
P1	Help alert (long clench)	5 second countdown, double blink cancels, otherwise places a call.
P1	Caregiver screen	Shows live signals (for judges), recent messages, settings, contacts and calibration.
P1	Yes / No mode	Caregiver turns it on. Clench = Yes, double blink = No, spoken aloud instantly.
P1	Body state reordering	Rising heart rate plus restlessness moves urgent options to the front.
P1	Rest pause	Eyes closed or no input for a set time pauses the highlight.
P2	Spell it	Letter groups with word guessing.
P2	Head tilt mode	Tilt moves the highlight using the headband's motion sensor.
P2	Room control	TV, lights, music through a mock smart home panel.
P2	Own voice	Speech in a cloned copy of the person's own voice (voice banking).
P2	Computer use	See Part 3.

8. The starting menu

This is the first version of the tree. The AI reorders each level. "Suggested" is always the first tile on the home screen.

Home tile	Next level	Example path and final sentence
Suggested	The AI's top 1 to 3 guesses for right now	Noon: "Tengo hambre. ¿Qué hay de almuerzo?"
I need	Water, Food, Bathroom, Pain, Move me, Too hot or cold	Pain → Back → A lot → "My back hurts a lot. Can you help me turn over?"
People	Maria, Carlos, Nurse (from contacts) → Call, Text, Say out loud	Maria → Text → "Mija, estoy bien, llámame a las seis."
How I feel	Good, Tired, Sad, Scared, Frustrated, Happy	Frustrated → "I'm frustrated. Give me a minute."
Say something	AI phrases from context, Spell it	"Thank you for today. I love you."
Room	TV, Lights, Music, Blinds	TV → On → "Please turn on the TV."

Why this tree: These six groups cover what caregivers report most: physical needs, reaching people, feelings and comfort. "Suggested" first means the most common case is one clench. "Help" is not a tile because it must work from any screen, so it is the long clench.

9. Learning the patient

We do not call this "self-improving AI." We say "it learns the patient," because that is what it does and everyone understands it.

Layer 1: learning their body

Calibration sets a starting line for what a clench and a blink look like. During use, the system keeps a moving record of the person's recent signals and slowly adjusts the line. When the person tires and clenches get weaker, the line drops with them.

Why: Fixed thresholds fail as the day goes on. This is the most useful learning for a real patient, and it is simple to build.

Layer 2: learning their habits

Every pick is logged: what, when, which path and whether it was confirmed or cancelled. Each option gets a score. In plain words:

```
score = how often they pick it (recent picks count more)
      + a bonus if they usually pick it at this time of day
      + a bonus if it fits their current body state
      + the AI's guess for this moment
      - a penalty if they recently rejected it (double blink)
```

Options are shown from highest score to lowest. Starting weights: use 0.4, time of day 0.2, body state 0.1, AI guess 0.3, tuned by hand. The AI also receives the person's most-used phrases, so it writes in their words ("Mija" rather than "daughter").

Why: This needs no model training. It is memory plus ranking, so it works from the first day and gets better with every use. A double blink after a suggestion is free feedback that the guess was wrong.

Layer 3: body state

See section 10. It only nudges the order.

Showing it in the demo

Learning cannot be seen unless we make it visible. We show "Day 1" (no history), where "text Maria" takes 5 steps. Then we load a week of practice history, and the same message takes 1 step. The screen shows **5** → **1**. We say plainly that the week of history is simulated, and that the ranking really uses it.

10. Reading body state

Signal	Sensor	How much we trust it	What we use it for
Jaw clench	Forehead and ear sensors (muscle noise)	High	Pick, long clench for help
Blink	Forehead sensors	High	Double blink for back and no
Eyes closed	Alpha waves (brain)	Medium to high	Rest pause
Heart rate	PPG light sensor	Medium (trend only)	Body state
Head movement	Accelerometer and gyroscope	High	Restlessness, head tilt mode
Where the face points	Laptop webcam (not the headband)	High in good light	Webcam pointing mode
"Tense vs calm"	Brainwave band ratios	Low	Small extra input to body state only

Body state has three levels: calm, normal, elevated. It changes only if the reading stays there for a while, so it does not flicker.

Why: Trusting each signal only as far as it deserves is what keeps the product honest and the demo reliable. Heart rate and movement carry most of the weight. The brainwave reading is a small extra, never the main input.

11. Safety, privacy and honesty

- **Confirm before anything leaves.** A final clench on "Send this?" every time.
- **Cancel window on help.** 5 second countdown so a yawn or chewing does not call anyone.
- **Local history.** Stored on the laptop. The AI gets only what it needs for the current guess.
- **Webcam video never leaves the laptop.** Face tracking runs inside the browser. No video is saved or sent anywhere, only "which tile is the person facing." A light on screen shows when the camera is on, and it can be turned off in settings at any time.
- **Honest pitch.** We say "we use jaw and blink signals, not mind reading." Judges who know the field will respect it. Overclaiming loses trust.
- **Not a medical device.** We say so on the caregiver screen and in the pitch.

12. How we measure success

Measure	Target for the demo
---------	---------------------

Steps to send a common message after learning	1 to 2 (from about 5 with no history)
Clenches caught (seated)	9 out of 10 or better
False picks while sitting still	1 or fewer per 5 minutes
Clench to spoken sentence (cached phrase)	Under 2 seconds
Demo reliability	Full script runs 3 times in a row with no restart

13. The demo

90-second script

1. **0:00** "Don Luis has ALS. He can't move or speak. He can still clench his jaw." A teammate sits still wearing the headband.
2. **0:15** He clenches. The projector shows the live signal spike and the board responding.
3. **0:30** People → Maria → Text. The AI offers three messages in Spanish. He picks one and confirms. The speaker says it and a judge's phone (or a team phone) buzzes.
4. **0:50** "It learns him." Switch to Day 1 vs after a week: 5 steps becomes 1. The counter shows it.
5. **1:00** Caregiver flips Pointing mode to Scan: "and if he loses neck control, it still works." The highlight starts moving by itself and he picks with a clench.
6. **1:10** He gets restless. Urgent options move to the front. He holds a clench: help countdown, then a call rings.
7. **1:20** Close: "Eye-gaze systems often cost thousands of dollars. This is a consumer headband and a laptop. And it speaks his language."

Questions judges will ask

Question	Answer
Is this reading his mind?	No. It reads jaw and blink signals, which are strong and reliable. That is a choice, not a limit we are hiding.
Why not a tablet or voice assistant?	Both need hands or a voice. This is for people with neither.
What if the AI says the wrong thing?	It cannot. Nothing is spoken or sent without his confirming clench.
What if he loses jaw control too?	Scanning works with any single reliable signal. The switch input can be swapped (for example to eyebrow raise or a physical switch) without changing the rest.
Where does his data go?	It stays on the laptop.

Bring: a Bluetooth speaker, a phone to receive texts, a phone hotspot, a charged headband and a recorded backup video.

14. Risks and backups

Risk	What we do
Muse arrives late or will not connect	The keyboard stand-in runs everything. As a truthful backup, a single large USB button is a real assistive switch, so scanning still works.
Signal is noisy in the hall (many Bluetooth devices)	Keep the laptop close. Recalibrate before judging. Record a clean session to replay for testing.
Talking or chewing triggers clenches	Rest pause and a short "ignore" window. Calibrate on the demo teammate.
Webcam loses the face (bad light, bright window behind, person turns away)	Auto mode falls back to Scan after about 3 s. Face the demo seat away from windows. Bring a small desk lamp.
Venue wifi is slow or blocks things	Use a phone hotspot. The menu works offline. Common phrases are cached with their audio.
AI is slow	Ask for suggestions the moment a menu opens, before the person reaches the end. Fall back to plain phrases.
Twilio trial limits	Trial accounts only text verified numbers. Verify the team's and demo phones in advance.
Judges think it is "mind reading" hype	Say what it is in the first 20 seconds.

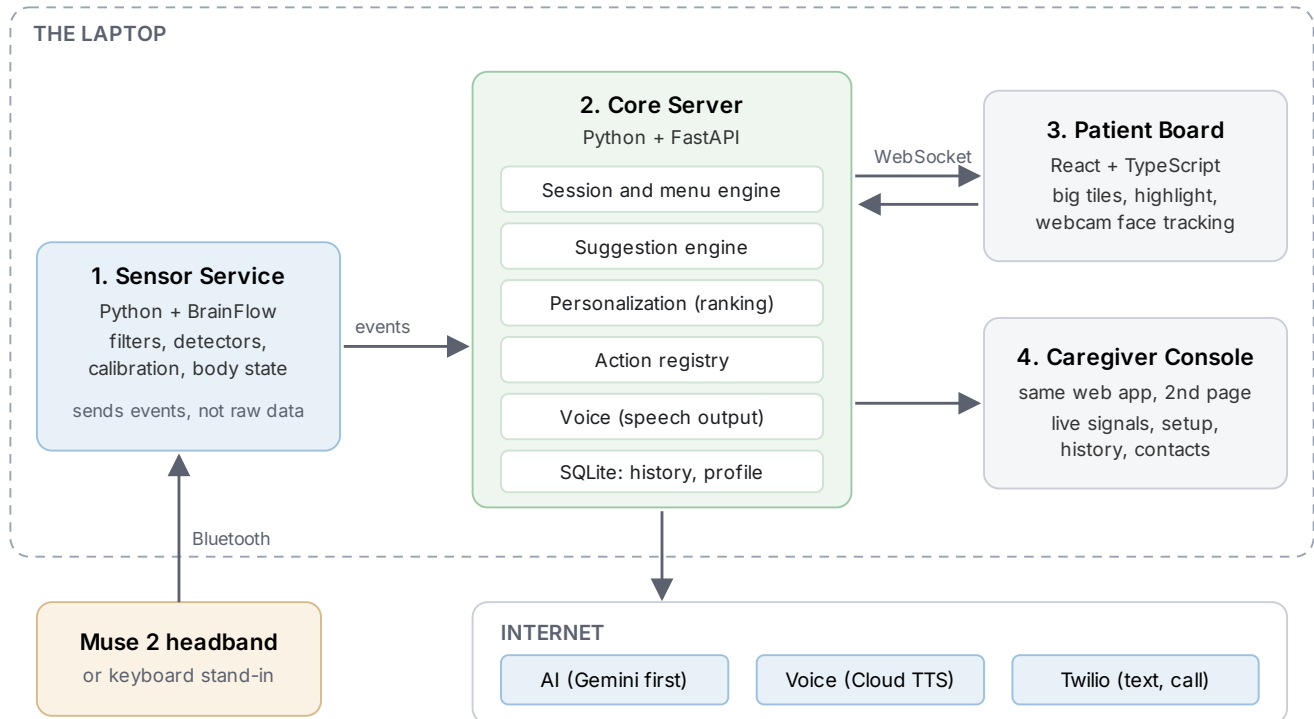
15. Open questions

- When exactly does the Muse arrive?
- What are this year's sponsor challenges? (Likely fits: Google Cloud agents, Gemini, Microsoft accessibility, Google social good.)
- Which Spanish and English voices sound most natural and warm?
- Do we show head tilt mode, or keep the demo to scanning only?
- Which phone numbers are used live on stage?

Planned Architecture

Four programs on one laptop, talking over local connections. Only the AI, the voice and the texting service reach the internet.

A1. The system at a glance



The one idea to remember: the Sensor Service turns messy signals into a few clean events (`CLENCH` , `DOUBLE_BLINK` , `LONG_CLENCH` , body state). Nothing else in the system knows or cares whether those events came from a real headband or the space bar.

A2. Stack choices and why

We did not pick the fastest option everywhere. We picked what is best for this product: reliable live input, low delay and a polished screen.

Layer	Choice	Why	Turned down
Headband connection	BrainFlow (Python)	Connects to the Muse 2 over the laptop's own Bluetooth on Windows, Mac and Linux. Its built-in <i>synthetic board</i> and <i>playback board</i> let us develop with fake or recorded data through the exact same code, then switch one ID when the Muse arrives. Also has filters, band power and a heart rate helper built in.	muselsl (fine, kept as backup, but no built-in fake board), Mind Monitor phone app (easy, but adds a phone and wifi hop)
Signal code	Python + NumPy / SciPy	Every EEG library and example is in Python. The team can read and change it fast.	JavaScript Web Bluetooth (few EEG tools, fragile)
Server	FastAPI (Python)	Same language as the signal code, so four people share one language on the back end. Built-in WebSocket support and typed data models. Good AI libraries in Python.	Node (would split the team across two back-end languages)
Separate sensor process	Yes , own program	Bluetooth can hang or crash. In its own process it can be restarted without losing the patient's place on the board. The keyboard stand-in is then just another program that sends the same events.	One big program (a Bluetooth hang freezes everything)
Live link	WebSocket	Two-way and instant: events go to the screen, picks come back. Easy to watch while debugging.	Polling (slow), gRPC (overkill)
Screen	React + TypeScript + Vite , Tailwind	A browser app runs full screen on any laptop or projector, looks polished, and changes appear instantly while coding. TypeScript catches mistakes in the event data. Vite is simpler than Next.js because we need no server pages.	Python desktop UI (looks dated, slow to style), Next.js (extra server we do not need), Electron (packaging work for no gain)
Webcam pointing	MediaPipe Face Landmarker (runs in the browser)	Free, runs on the laptop in real time with no server, and gives head angle directly. Running it in the browser means video never leaves the machine and there is no video to send between programs, only a small "facing tile 3" message.	OpenCV in Python (would have to stream video to the server), cloud vision APIs (slow, and sends video of a patient to the internet)

Live signal chart	uPlot	Draws fast-moving data smoothly, which matters for the judges' view of clench spikes.	Chart.js / Recharts (slow with streaming data)
AI	Gemini via the Google GenAI SDK, behind our own interface	A fast Flash-tier model keeps suggestions quick. Structured output returns clean JSON. Fits Google and MLH Gemini sponsor prizes. Our interface means swapping to Claude or OpenAI is one setting.	Hard-coding one provider
Voice	Google Cloud Text-to-Speech , browser speech as fallback	Natural voices in US English and US Spanish. We save the audio for common phrases so they play instantly. The browser's built-in voice works offline as a backup.	Browser voice only (robotic Spanish)
Texts and calls	Twilio	Real texts and real calls on stage with a few lines of code.	WhatsApp API (slow approval; Twilio's WhatsApp sandbox is a P2 option)
Storage	SQLite	One local file, no setup, keeps health data on the laptop.	Cloud database (privacy risk, setup time)

A3. The parts in detail

1. Sensor Service

Its only job is to turn signals into events. It has four swappable **sources**, all behind one interface:

Source	Use
<code>KeyboardSource</code>	Space = clench, hold space = long clench, B B = double blink, H = raise heart rate. For UI and logic work with no headband.
<code>SyntheticSource</code>	BrainFlow's synthetic board plus fake clench bursts we add in. Tests the full signal pipeline before the Muse arrives.
<code>ReplaySource</code>	Plays back a recorded CSV session. Once we record the real Muse, every test can rerun the same data.
<code>MuseSource</code>	The real Muse 2 through BrainFlow (<code>BoardIds.MUSE_2_BOARD</code>), with EEG, motion and PPG presets.

The pipeline inside: **filter** the raw signal → **detect** clench and blink → **debounce** so one clench is one event → **estimate body state** → **send events**. It also runs the calibration steps when the Core asks, and streams a thinned-out copy of the signal for the caregiver chart.

2. Core Server

- **Session and menu engine.** A simple state machine: `IDLE`, `SCANNING`, `CONFIRMING`, `SPEAKING`, `HELP_COUNTDOWN`, `PAUSED`, `CALIBRATING`. It owns the scan timer, so the highlight position is always known in one place.
- **Suggestion engine.** When a menu opens, it immediately asks the AI for this level's ranking and sentences, so results are ready before the highlight finishes one pass. Results are cached.
- **Personalization.** Logs every pick, computes the score from section 9, and sorts options.
- **Action registry.** Each action is a small tool with a name, inputs and a "needs confirm" flag: `speak`, `send_text`, `place_call`, `room_control`, `help_alert`. The same registry is used later by computer use (Part 3).
- **Voice.** Turns sentences into audio, saves common ones, sends audio to the board to play.

3. Patient Board

Full screen, high contrast, huge text. The board draws whatever the Core tells it. It sends back only three things: "ready," audio status, and, in Webcam mode, which tile the person is facing. All decisions stay in the Core.

Why keep the board "dumb": one source of truth for where the highlight is. If the board and server both decided it, they would drift apart and a clench would pick the wrong thing. In Webcam mode the board only reports "facing tile 3." The Core still decides what is highlighted.

3a. Pointing modes (part of the Core, with a webcam helper in the board)

The Core has one **pointer** slot that holds the current mode. Each mode is small and does one thing: tell the Core which tile should be highlighted.

Mode	Where it runs	What it does
Scan	Core (scan timer)	Moves the highlight one tile every set number of ms (default 1000, adjustable).
Webcam	Browser (MediaPipe) → Core	Face Landmarker gives head turn and tilt angles. The board maps the angle to a tile, smooths it and sends <code>POINT</code> only when the tile changes.
Head tilt	Sensor Service → Core	Reads tilt from the headband's motion sensor and sends <code>POINT</code> the same way.
Auto	Core	Uses Webcam while <code>FACE_OK</code> is true. If no face for about 3 s, switches to Scan and shows a small "scanning" badge. Switches back when the face returns.

Making webcam pointing steady:

- **Calibrate center and edges.** During setup, the person looks at the middle, then as far left and right as is comfortable. Tiles are spread across *their* range, so a small movement is enough.
- **Smooth the angle** over a few frames so shaking does not jump tiles.

- **Sticky edges.** The highlight only moves to a new tile once the head has clearly crossed into it, which stops flicker on the border.
- **Freeze on clench.** A clench moves the jaw and can shift the head slightly. When a clench starts, lock the highlight for about 300 ms so the right tile gets picked.

Why one slot with swappable modes: it is the same trick as the headband stand-in. The rest of the system only sees "highlight tile 3" and "clench," so adding or switching a mode touches nothing else. It also makes the setting a single value the caregiver changes on the console.

4. Caregiver Console

A second page of the same web app. A **Pointing mode** switch (Auto / Scan / Webcam / Head tilt) and scan speed slider, a small webcam preview with the tracked face when Webcam is on, live signal chart with detected events marked, calibration wizard, contacts, language, scan speed, message history, the "5 → 1" learning counter, and a switch to load demo history. This is also the judges' window into what the headband sees.

5. AI provider layer

```
class LLMProvider(Protocol):
    def rank_and_suggest(self, ctx: SuggestContext) → SuggestResult: ...

# ctx: current path, language, time of day, body state,
#      top 20 most-used phrases, last 5 messages, contacts
# result (JSON): ranked option ids + up to 3 sentences,
#      each with text, language and confidence

PROVIDER = env("LLM_PROVIDER", "gemini") # gemini | claude | openai
```

Use low randomness so suggestions are steady. Put the phrasing rules in the prompt: short sentences, first person, use the patient's own words, match the chosen language.

A4. How the parts talk

All messages are small JSON objects over WebSocket.

```
// Sensor Service → Core
{"type":"CLENCH","t":1727300000.12,"strength":0.83}
{"type":"DOUBLE_BLINK","t":1727300003.40}
{"type":"LONG_CLENCH","t":1727300009.90,"duration":1.6}
{"type":"STATE","t":...,"level":"elevated","hr":94,"motion":0.7,"eyes_closed":false}
{"type":"SIGNAL","t":...,"ch":[...]} // thinned, for the chart only

// Board → Core (Webcam mode only)
{"type":"POINT","source":"webcam","tile":3,"t":...}
{"type":"FACE_OK","ok":false} // face lost, Auto falls back to Scan

// Console → Core
{"type":"SETTINGS","pointing_mode":"auto","scan_ms":1000}

// Core → Board
{"type":"SCREEN","screen":"menu","tiles":[{"id":"people","label":"People"}, ...],
  "highlight":2,"lang":"es"}
{"type":"CONFIRM","text":"Mija, estoy bien, llámame a las seis.,"action":"send_text"}
{"type":"PLAY_AUDIO","url":"/audio/abc123.mp3"}
```

Why write this down first: once the event shapes are agreed, the signal work, the server, and the screen can be built at the same time by different people, and they will fit together.

A5. Signal processing

These are starting values. Every number gets tuned on the real headband after calibration.

Jaw clench

A clench floods the sensors with muscle noise at high frequencies. Filter each channel to about 30 to 100 Hz, measure its strength over short windows (about 100 ms), and compare to the calm baseline from calibration. A clench starts when the strength stays above the threshold for about 80 ms. After a clench, ignore new ones for about 400 ms. If it stays above the threshold for about 1.5 s, it is a long clench.

Blink

A blink makes a large, slow swing on the two forehead sensors at the same time. Filter to about 0.5 to 10 Hz and look for a swing larger than the calibrated size that lasts 100 to 400 ms on both forehead channels. Two blinks within about 700 ms make a double blink. Single blinks are ignored.

Thresholds that adjust

Calibration records rest, clenches and blinks, then sets each threshold between the rest level and the typical clench level. During use, a slow moving average of confirmed clench strength nudges the threshold up or down.

Body state

Heart rate from the PPG sensor using BrainFlow's heart rate helper (check the Muse 2 PPG channel layout first). Restlessness from how much the motion sensor readings vary. Eyes closed from a rise in alpha power (8 to 12 Hz). Combine them into calm, normal or elevated, and only change level after it holds for several seconds.

Honest note: the Muse picks up muscle and eye movement much more strongly than brain activity. For our purpose that is good news: clench and blink are exactly the strong signals. Do not spend hours making brainwave readings "accurate." They are a small extra.

A6. Speed budget

Step	Target
Clench happens → event sent	Under 250 ms
Event → screen updates	Under 50 ms
Head turns → highlight moves (Webcam mode)	Under 150 ms
Menu opens → AI ranking ready	Under 1.5 s (fetched in the background while scanning)
Confirm → voice starts	Instant for saved phrases, under 1.5 s for new ones
Confirm → text arrives on phone	A few seconds (Twilio)

A7. Data we store

```
profile(id, name, language, pointing_mode, scan_speed_ms, thresholds_json,
        head_range_json, created_at)
contacts(id, profile_id, name, relation, phone, language)
menu_nodes(id, parent_id, label_en, label_es, kind, action) -- the tree
events(id, profile_id, t, node_id, path, action, confirmed, rejected, state_level)
phrases(id, profile_id, text, lang, uses, last_used, hour_histogram_json)
audio_cache(hash, text, lang, voice, file_path)
sessions(id, profile_id, started_at, source, calibration_json)
```

A `seed_demo_week.json` file fills `events` and `phrases` with a believable week of use for the "5 → 1" moment.

A8. Code layout

```
clench/
  sensor/          # Sensor Service (Python)
    sources/       #   keyboard.py, synthetic.py, replay.py, muse.py
    detect/        #   clench.py, blink.py, state.py, calibrate.py
    main.py        #   picks a source from settings, sends events
  core/            # Core Server (FastAPI)
    session.py     #   state machine
    pointer/       #   scan.py, webcam.py, headtilt.py, auto.py
    menu.py        #   tree loading and walking
    suggest/       #   provider.py (interface), gemini.py, claude.py, openai.py
    rank.py        #   personalization score
    actions/       #   speak.py, sms.py, call.py, room.py, help.py
    voice.py       #   TTS and audio cache
    db.py
  web/             # React + TypeScript + Vite
    src/board/     #   patient screen
    src/facetrack/ #   MediaPipe Face Landmarker, angle to tile
    src/console/   #   caregiver screen and live chart
  data/
    menu.yaml      #   the menu tree, both languages
    seed_demo_week.json
    recordings/     #   CSVs of real Muse sessions
    .env.example   #   API keys, LLM_PROVIDER, SOURCE=keyboard|synthetic|replay|muse
```

A9. Build plan: before and after the Muse

Four workstreams that can run at the same time. Owners are not assigned yet.

Workstream	Before the Muse arrives	After the Muse arrives
Signals	Keyboard source. Synthetic and replay sources with fake clench bursts. Clench and blink detectors tested on synthetic data. Calibration flow.	Connect <code>MuseSource</code> . Record 10 minutes of real data (rest, clenches, blinks, talking, chewing). Tune thresholds. Save recordings for replay tests.
Core	State machine, menu tree, scan timer, WebSocket, action registry with SMS and speech, SQLite.	Tune scan speed and debounce with a real person. Help alert timing.
AI and learning	Provider interface, Gemini prompts and JSON output, ranking score, demo week seed, prefetch and cache.	Feed real body state into ranking. Check suggestions in both languages.
Screens	Patient board, confirm screen, caregiver console, live chart fed by synthetic data, "5 → 1" counter. Webcam pointing with MediaPipe and the Pointing mode switch (fully testable now: webcam plus space bar as the clench).	Polish with the real signal chart. Rehearse the demo.

First milestone (everyone): press space, the board highlight picks a tile, the laptop speaks a phrase. That proves every connection end to end. Build out from there.

A10. Testing

- **Detector tests:** run each recorded session through the detectors and count hits and false picks. Rerun after every threshold change.
 - **Script tests:** a script of keyboard events that walks the full demo path, so we can check the whole app in seconds.
 - **Offline test:** turn off wifi and confirm the menu, speech from cache and help alert still work.
 - **Demo drills:** run the full 90 seconds three times in a row with the real headband.
-

A11. Where to learn

Muse and EEG

- BrainFlow documentation (start here): brainflow.readthedocs.io
- BrainFlow supported boards, Muse section and board IDs: [SupportedBoards.html](#)
- BrainFlow code samples (Python streaming, filters, band power): [Examples.html](#)
- BrainFlow API reference (DataFilter, heart rate, band power): [UserAPI.html](#)
- BrainFlow source code: github.com/brainflow-dev/brainflow
- muselsl (backup way to stream the Muse): github.com/alexandrebarachant/muse-lsl
- Muse 101, a beginner walkthrough of Muse 2 development: [Medium](#), [Anush Mutyala](#)
- Building a bedroom BCI, collecting Muse 2 data: [Medium](#), [Tim de Boer](#)
- Blink-controlled Pong with a Muse (practical blink detection notes): github.com/baljo/Muse-EEG
- brain-beat, Muse 2 project with muscle-noise handling and PPG: github.com/freakandstein/brain-beat
- Mind Monitor app (quick way to see live signals on a phone) and FAQ: musemonitor.com/FAQ.php
- Mind Monitor forum on blink and jaw clench events in Python: [forum thread](#)
- NeuroTechX EEG-ExPy, open experiments built for Muse: github.com/NeuroTechX/EEG-ExPy
- Lab Streaming Layer docs (if using muselsl): labstreaminglayer.readthedocs.io
- More Muse projects to borrow from: github.com/topics/muse-headband

AI, voice and actions

- Gemini API docs: ai.google.dev/gemini-api/docs
- Gemini structured output (JSON replies): [structured-output](#)
- Google GenAI Python SDK: github.com/googleapis/python-genai
- Google Cloud Text-to-Speech: cloud.google.com/text-to-speech/docs
- Twilio SMS with Python: twilio.com/docs/messaging/quickstart/python
- FastAPI WebSockets: fastapi.tiangolo.com/advanced/websockets
- MediaPipe Face Landmarker for web (head angle from the webcam): ai.google.dev/edge/mediapipe/.../face_landmarker/web_js

- Vite: vite.dev/guide · uPlot: github.com/leeoniya/uPlot

Background on the problem

- ALS Association on communication tools (AAC), switch scanning and eye gaze: als.org
- ALS Association, Communication Options for People with ALS (PDF): als.org PDF

Computer use with the Muse

A new home tile, *Do it for me*, lets the person run tasks on the computer. They pick the task with the same clench and blink. An AI agent does the clicking, and the person approves each important step with a clench.

B1. What it does

Example: *Do it for me* → *Play music* → *Celia Cruz*. The agent opens a browser, finds a playlist and plays it. Or: *Email* → *Dr. Pérez* → *"Refill my prescription"*. The agent writes the email, shows it, and waits for a clench before sending.

B2. Two ways to do it

Approach	How it works	Good	Bad
A. Scanning mouse	The screen is split into a grid. The highlight scans rows, then columns, then smaller squares. Clench to click.	Full control, no AI, works on anything.	Very slow. One click can take 10 or more clenches.
B. AI agent with approval	The person picks a task from a short menu. The agent does the clicks. Before anything that sends, buys or deletes, it pauses for a clench.	Fast. One or two clenches for a whole task. Great demo.	Can make mistakes. Needs internet.

Decision: build B, keep A as a later fallback. For someone who can only clench, a task-level agent turns minutes of clicking into seconds. The approval gate keeps the person in charge, which is the same rule as D5. A is the right safety net for a real product but will not impress in a demo.

B3. Scope

In: three tasks built well: play music or a video, write and send an email to a saved contact, look something up and read the answer aloud (for example the weather or a hurricane update). A fixed allowlist of websites. A separate browser profile just for the agent.

Out: payments, shopping, account settings, any site not on the allowlist, and free-form "do anything" tasks.

Why only three tasks: the teams that won with computer-use agents gave them purpose-built tools for their demo tasks instead of letting them click freely. Three tasks that work every time beat twenty that work half the time.

B4. How it plugs in

```
Board: "Do it for me" → pick task → pick details
  → Core: action "run_task" (needs confirm)
  → Agent Runner (new, Python)
    - plans steps with Gemini
    - uses our own tools first: open_youtube(query), send_email(to, body),
      web_answer(question)
    - falls back to Gemini's Computer Use tool for open-ended steps:
      screenshot → model suggests a click or typing → Playwright does it → repeat
    - any step flagged as risky (send, delete, submit)
      → Core shows "Approve this step?" → CLENCH approves, DOUBLE_BLINK stops
  → result spoken aloud and shown on the board
```

- **Browser control:** Playwright running its own Chromium with a separate profile, so the agent never touches the person's main accounts unless we sign it in on purpose.
- **Model:** Gemini's Computer Use tool analyzes screenshots and returns actions for browser, mobile or desktop, and it can flag sensitive actions that need user confirmation. We map that confirmation straight to the clench. Check the docs for the current model name.
- **Agent framework:** Google's Agent Development Kit (ADK) is optional but fits the Google Cloud agent prize if it returns this year.
- **Same action registry:** tasks are just new tools in the registry from Part 2, so confirm, logging and learning all work for free. Clench will learn which tasks the person runs most.

B5. Rough effort

Piece	Rough time
Playwright setup and 3 purpose-built tools	4 to 6 hours
Approval gate wired to clench and blink	1 to 2 hours
Gemini Computer Use loop for open-ended steps	3 to 5 hours
Board screens for tasks and approvals	2 hours

Start rule: do not start this until every P0 feature works with the real headband and the demo has been run end to end once.

B6. Where to learn

- Gemini Computer Use: ai.google.dev/gemini-api/docs/computer-use
- Playwright for Python: playwright.dev/python
- Google Agent Development Kit (ADK): google.github.io/adk-docs